



UNIVERSIDAD ARGENTINA DE LA EMPRESA

Algoritmos y Estructura de Datos I

◆
Sistema de Gestión de Micros

Docentes

Lic. Gustavo Escandell

Ing. Melinda Selles

Integrantes y Roles

- Velasquez, Lorena (1146019) – Desarrolladora
- Astudillo, Valentina (1211931) – Analista
- Chaina, Brian (1213338) – Tester
- Ayala, Alan (1196870) – Diseñador de UI/UX
- Farfan, Mauricio (1167832) – Líder del Proyecto

Fecha de Entrega

22 de abril de 2026

2026

Índice

1. Presentación de la Propuesta.....	3
1.1 Introducción.....	3
1.2 Problemática.....	3
1.3 Solución.....	3
2. Alcance del Sistema.....	4
2.1 Lo que el Sistema Puede Hacer.....	4
2.2 Lo que el Sistema NO Hace (Fuera de Alcance).....	4
3. Funcionalidades Implementadas.....	5
Opción 1: Buscar Viaje.....	5
Opción 2: Comprar Pasaje.....	5
Opción 3: Ver Recaudación.....	5
Opción 4: Salir.....	5
4. Explicación Técnica de Herramientas.....	6
4.1 Listas Avanzadas: El Catálogo de Viajes.....	6
4.2 Matrices: Mapa de Asientos.....	7
4.3 Funciones Lambda.....	7
4.4 Programación Funcional: filter(), map() y reduce().....	8
4.5 Expresiones Regulares (RegEx).....	9
5. Documentación del Sistema.....	12
5.1 Estructura de Módulos.....	12
5.2 Flujo Principal del Sistema.....	13
5.3 Casos Borde Contemplados.....	14
5.4 Guía de Uso para el Usuario.....	14
5.5 Guía de Resolución de Errores.....	15
6. Conclusión.....	15

1. Presentación de la Propuesta

1.1 Introducción

El presente trabajo práctico presenta **Nexus Viajes**, un sistema de gestión de micros diseñado para optimizar la comercialización de boletos y la asignación de butacas en tiempo real. El foco principal es centralizar la operación en un nexo tecnológico que reemplace los procesos manuales, eliminando errores de asignación y sobreventas mediante una herramienta eficiente y escalable para la administración de datos.

1.2 Problemática

Las agencias de transporte de media y larga distancia necesitan gestionar, en tiempo real, la disponibilidad de asientos, la venta de pasajes y el registro de la recaudación diaria. Sin un sistema centralizado, estos procesos se realizan de forma manual, lo que genera errores en la asignación de asientos, pérdida de información financiera y demoras en la atención al pasajero.

1.3 Solución

El sistema propuesto digitaliza el flujo completo de venta: desde la consulta de viajes disponibles hasta la emisión del ticket y el cómputo de la recaudación. Se eligió este caso de uso porque:

- Permite aplicar de forma natural todos los conceptos obligatorios de la materia (listas, matrices, funciones lambda, RegEx).
- Refleja un escenario real y relevante dentro del ámbito de la transformación digital.
- El flujo lineal del proceso de compra facilita la modularización clara y ordenada del código.

2. Alcance del Sistema

2.1 Lo que el Sistema Puede Hacer

El alcance de esta primera etapa comprende el desarrollo del "Motor de Ventas" (Core Business) de la terminal de ómnibus. El sistema está diseñado para operar mediante una interfaz de línea de comandos (CLI) y procesa toda la información en la memoria volátil del sistema (RAM). Abarca desde la consulta inicial del cliente hasta la emisión del ticket y el arqueo de caja de la sucursal, cumpliendo con las restricciones de no utilizar bases de datos ni persistencia en archivos para esta instancia.

- Buscar viajes disponibles filtrando por origen, destino y fecha.
- Mostrar visualmente el mapa de asientos de cada micro (matriz 3x4).
- Permitir al pasajero seleccionar un asiento libre e ingresar sus datos personales.
- Calcular automáticamente el precio final aplicando el recargo del 16% por servicio.
- Registrar cada venta en memoria y mostrar la recaudación total del día mediante `reduce()`.
- Validar todos los inputs del usuario mediante expresiones regulares (DNI, email, teléfono, fecha).
- Permitir cancelar la operación en cualquier momento durante la selección de asiento.

2.2 Lo que el Sistema NO Hace (Fuera de Alcance)

- No persiste datos entre ejecuciones (no usa archivos ni base de datos).
- No admite múltiples pasajeros por transacción ni lógica de carrito de compras.
- No gestiona devoluciones ni cancelaciones de pasajes ya emitidos.
- No tiene interfaz gráfica; opera completamente por consola.
- No implementa autenticación de usuarios ni roles (administrador / cliente).

3. Funcionalidades Implementadas

El sistema se controla a través de un Menú Principal interactivo que ofrece 4 flujos operativos distintos:

Opción 1: Buscar Viaje

El usuario ingresa el origen, el destino y la fecha del viaje. El sistema utiliza `filter()` con una función lambda para comparar esos datos contra el catálogo en memoria y devuelve todos los viajes que coinciden exactamente. El resultado se presenta en formato tabular con ID, empresa, fecha y precio base. Si no hay coincidencias, se muestra un mensaje de error descriptivo.

Opción 2: Comprar Pasaje

Es el flujo principal del sistema y consta de cuatro pasos:

- Paso A: Selección de viaje: el usuario ingresa el ID del viaje deseado. Si el ID no existe en el catálogo, se informa el error y se vuelve al menú sin `crash`ear.
- Paso B: Selección de asiento: se muestra el mapa visual de la matriz 3x4 con el estado de cada asiento (L = Libre, O = Ocupado). El usuario elige fila y columna; puede ingresar 'o' en cualquier momento para cancelar. Se validan los límites de la matriz y la disponibilidad del asiento antes de continuar.
- Paso C: Registro del pasajero: se solicita DNI, email y teléfono, validados en tiempo real con expresiones regulares. Si el dato es inválido, el sistema lo informa y vuelve a pedirlo sin cortar la ejecución.
- Paso D: Liquidación y emisión: se calcula el precio final con `map()` aplicando el recargo del 16%. Se muestra un resumen de compra y, si el usuario confirma, se emite el ticket, se actualiza la matriz (L → O) y se registra la venta en la lista `ventas_diarias`.

Opción 3: Ver Recaudación

Muestra un reporte de caja con la cantidad de pasajes vendidos en la sesión y el total recaudado. Este cálculo se realiza mediante `reduce()` sobre la lista `ventas_diarias`. Si no se realizaron ventas, el sistema maneja correctamente la lista vacía devolviendo \$0.00 en lugar de lanzar un `TypeError`.

Opción 4: Salir

Finaliza la ejecución del programa mostrando un mensaje de despedida y liberando los recursos en memoria.

4. Explicación Técnica de Herramientas

4.1 Listas Avanzadas: El Catálogo de Viajes

Toda la información del sistema se almacena en memoria usando listas y listas anidadas, sin diccionarios ni bases de datos. El catálogo principal es una lista de listas donde cada sub-lista representa un viaje con índices fijos:

Índice	Campo	Tipo	Ejemplo
[0]	ID del viaje	String	"1"
[1]	Empresa	String	"Via Bariloche"
[2]	Origen	String	"Buenos Aires"
[3]	Destino	String	"Bariloche"
[4]	Fecha	String	"20/05"
[5]	Precio base	Float	50000.0
[6]	Matriz asientos	Lista 2D	[["L","O"],["L","L"]]

Este diseño permite acceder a cualquier dato de forma directa por índice (viaje[1] para empresa, viaje[5] para precio) y recorrer el catálogo completo con un simple bucle for o con filter().

```
8  catalogo_viajes = [  
9      [  
10         "1",  
11         "Via Bariloche",  
12         "Buenos Aires",  
13         "Bariloche",  
14         "20/05",  
15         50000.0,  
16         [  
17             ["L", "L", "L", "L"],  
18             ["L", "O", "L", "L"],  
19             ["O", "O", "L", "L"]  
20         ]  
21     ],
```

4.2 Matrices: Mapa de Asientos

Cada viaje incluye una lista bidimensional (matriz 3x4) que representa el interior del micro: 3 filas de asientos y 4 columnas. Cada celda contiene el string "L" (Libre) u "O" (Ocupado).

```
matriz_asientos = [  
    ["L", "L", "L", "L"], # Fila 1  
    ["L", "O", "L", "L"], # Fila 2  
    ["O", "O", "L", "L"]  # Fila 3  
]
```

Para verificar si un asiento está libre se accede con `matriz[filas - 1][columna - 1]` (se resta 1 porque el usuario piensa en base 1 pero Python indexa en base 0). Para reservarlo, simplemente se cambia el valor a "O". La función `mostrar_micro()` recorre la matriz con `enumerate()` para imprimir el mapa visual en consola.

4.3 Funciones Lambda

Las funciones lambda son funciones anónimas de una sola expresión. En el sistema se usan en tres contextos obligatorios:

Función	Lambda usada	Qué hace
<code>buscar_viajes()</code>	<code>lambda viaje: viaje[2].lower() == origen...</code>	Filtra viajes por origen, destino y fecha
<code>buscar_por_id()</code>	<code>lambda viaje: viaje[0] == id_viaje</code>	Busca un viaje específico por su ID
<code>aplicar_recargo()</code>	<code>lambda p: p * 1.16</code>	Aplica el 16% de recargo al precio base
<code>calcular_recaudacion()</code>	<code>lambda a, b: a + b</code>	Suma acumulativamente los precios de venta

4.4 Programación Funcional: filter(), map() y reduce()

1. filter(): Módulo d_búsquedas.py

Recorre el catálogo completo y devuelve únicamente los viajes que cumplen la condición de la lambda. Se convierte el resultado a lista con list() para poder consultar su longitud y acceder a sus elementos.

```
1 def buscar_viajes(origen_buscado, destino_buscado, fecha_buscada):
2
3     viajes_filtrados = list(filter(
4         lambda viaje: viaje[2].lower() == origen_buscado.lower() and
5         viaje[3].lower() == destino_buscado.lower() and
6         viaje[4] == fecha_buscada,
7         catalogo_viajes
8     ))
9
10    return viajes_filtrados
```

2. map(): Módulo e_finanzas.py

Transforma el precio base aplicando el 16% de recargo. Se llama con una lista de un solo elemento y luego se extrae el primer resultado, respetando la semántica de map() que opera sobre colecciones.

```
19 def aplicar_recargo(precio_base):
20
21     precio_con_recargo = list(map(lambda p: p * 1.16, [precio_base]))
22     return precio_con_recargo[0]
```

3. reduce(): Módulo e_finanzas.py

Suma todos los precios de ventas_diarias de forma acumulativa. Se maneja el caso borde de lista vacía con un condicional previo para evitar que reduce() lance un TypeError.

```
15 def calcular_recaudacion_total(lista_ventas):
16
17     if len(lista_ventas) == 0:
18         return 0.0
19
20     return reduce(lambda a, b: a + b, lista_ventas)
```


4.5 Expresiones Regulares (RegEx)

Se usa la librería `re` de Python con `re.match()` para validar cada input antes de procesarlo. Esto evita el uso de bloques `try/except` y garantiza que los datos tengan el formato correcto antes de continuar con el flujo.

Campo	Patrón RegEx	Descripción
Fecha	<code>^\d{2}/\d{2}\$</code>	Exactamente DD/MM: dos dígitos, barra, dos dígitos
DNI	<code>^\d{7,8}\$</code>	Solo dígitos, entre 7 y 8 caracteres
Email	<code>^[\w\.-]+@[\w\.-]+\.\w+\$</code>	Formato básico: usuario@dominio.extensión
Teléfono	<code>^\d{8,15}\$</code>	Solo dígitos, entre 8 y 15 caracteres

Además de la validación de formato, la función `validar_fecha()` realiza una validación lógica adicional: verifica que el día no supere el máximo permitido para cada mes (usando una lista `días_maximos` indexada por número de mes).

A continuación se muestra como se implementó en el código.

1. Función `validar_fecha()`:

```
3 def validar_fecha(fecha_ingresada):
4
5     """
6     Valida la fecha ingresada por el usuario.
7     Asegura que el formato sea DD/MM y que el día y mes sean válidos.
8     """
9
10    dias_maximos = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
11
12    while True:
13        # RegEx: Formato DD/MM, donde DD es 01-31 y MM es 01-12
14        if re.match(r'^\d{2}/\d{2}$', fecha_ingresada):
15            dia, mes = map(int, fecha_ingresada.split('/'))
16
17            if 1 <= mes <= 12:
18                if 1 <= dia <= dias_maximos[mes - 1]:
19                    return fecha_ingresada
20
21        print("\n -----")
22        print(" [ ERROR ] Fecha inválida. Ingresela en formato DD/MM. ")
23        print(" -----\n")
24        fecha_ingresada = input("> Ingrese la fecha del viaje (DD/MM): ")
```

2. Función validar_dni():

```
27 def validar_dni(dni_ingresado):
28
29     """
30     Valida el DNI ingresado por el usuario.
31     Asegura que el DNI contenga solo números y tenga entre 7 y 8 dígitos.
32     """
33
34     while True:
35         # RegEx: Exactamente entre 7 y 8 dígitos numéricos
36         if re.match(r'^\d{7,8}$', dni_ingresado):
37             return dni_ingresado
38         print("\n -----")
39         print(" [ ERROR ] El DNI debe contener 7 u 8 dígitos. ")
40         print(" ----- \n")
41         dni_ingresado = input("> Ingrese su DNI (sin puntos): ")
42
```

3. Función validar_email():

```
44 def validar_email(email_ingresado):
45
46     """
47     Valida el email ingresado por el usuario.
48     Asegura que el email tenga un formato básico de email@dominio.com
49     """
50
51     while True:
52         # RegEx: Formato basico de email texto@texto.texto
53         if re.match(r'^[\w\.-]+@[\w\.-]+\.\w+$', email_ingresado):
54             return email_ingresado
55         print("\n -----")
56         print(" [ ERROR ] Formato de email inválido. ")
57         print(" ----- \n")
58         email_ingresado = input("> Ingrese su Email: ")
59
```

4. Función validar_telefono():

```
61 def validar_telefono(tel_ingresado):
62
63     """
64     Valida el teléfono ingresado por el usuario.
65     Asegura que el teléfono contenga solo números y tenga entre 8 y 15 dígitos.
66     """
67
68     while True:
69         # RegEx: Solo números, entre 8 y 15 dígitos
70         if re.match(r'^\d{8,15}$', tel_ingresado):
71             return tel_ingresado
72         print("\n -----")
73         print(" [ ERROR ] Teléfono inválido (solo números). ")
74         print(" -----\n")
75         tel_ingresado = input("> Ingrese su Teléfono: ")
```

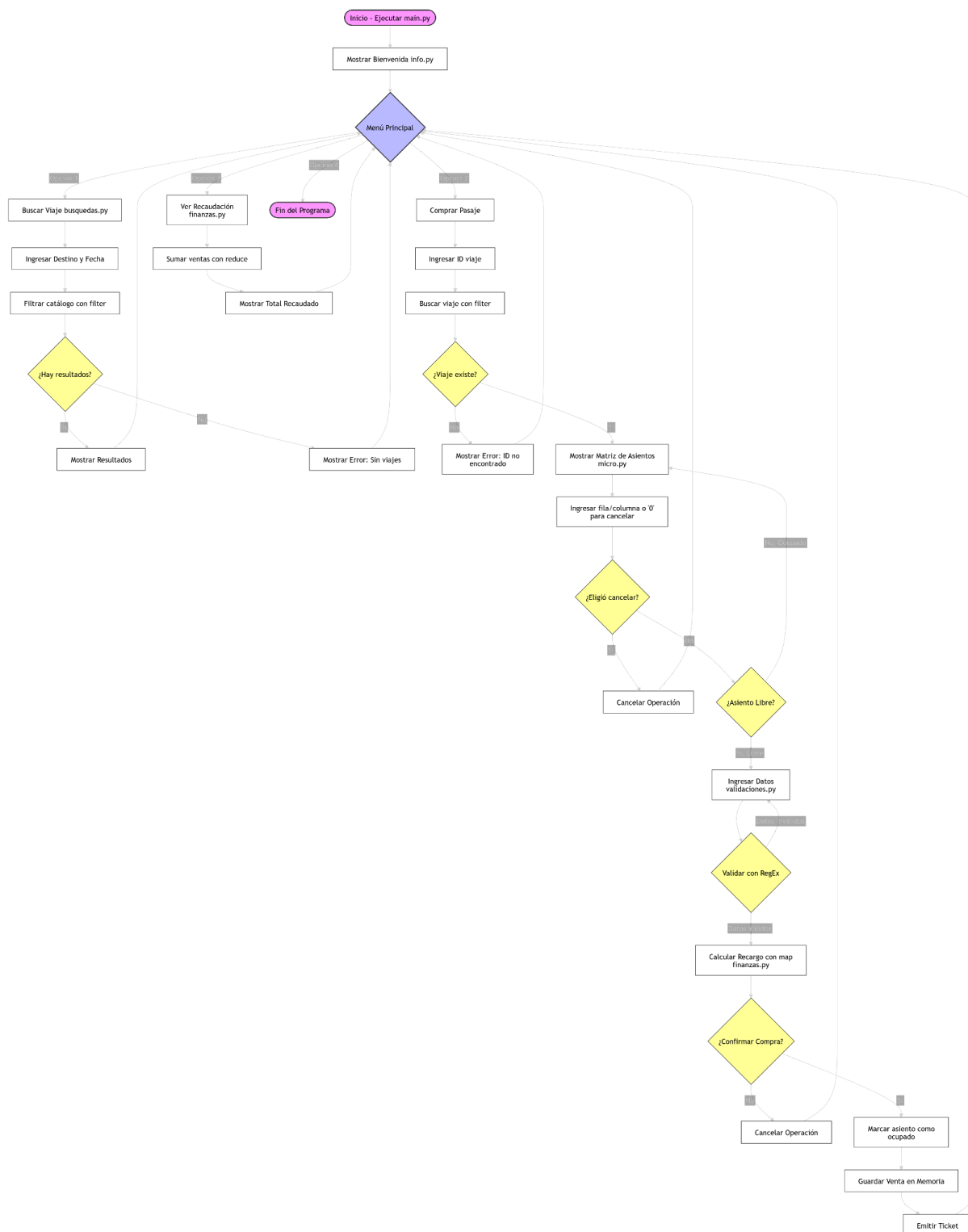
5. Documentación del Sistema

5.1 Estructura de Módulos

Archivo	Responsabilidad Principal
a_main.py	Punto de entrada. Contiene el menú principal (while True) y orquesta el flujo de compra completo.
b_info.py	Muestra la pantalla de bienvenida con los datos estáticos de la agencia.
c_micro.py	Imprime el mapa visual de asientos, verifica disponibilidad y reserva asientos ($L \rightarrow O$).
d_busquedas.py	Almacena el catálogo de viajes (hardcodeado) y expone las funciones de búsqueda con filter().
e_finanzas.py	Calcula el precio final con map() y computa la recaudación total con reduce().
f_validaciones.py	Valida todos los inputs del usuario mediante expresiones regulares (re.match).

5.2 Flujo Principal del Sistema

El siguiente diagrama describe el flujo de ejecución del programa:



5.3 Casos Borde Contemplados

Situación	Cómo se maneja
ID de viaje inexistente	buscar_por_id() devuelve None; se muestra error y se vuelve al menú con continue.
Lista vacía en reduce()	calcular_recaudacion_total() verifica len == 0 y devuelve \$0.00 antes de llamar a reduce().
Asiento fuera de rango	Se valida $1 \leq \text{fila} \leq 3$ y $1 \leq \text{columna} \leq 4$ antes de acceder a la matriz.
Asiento ya ocupado	asiento_esta_libre() devuelve False; se muestra error y se repite la selección.
Input no numérico en asiento	Se valida con .isdigit() antes de convertir a int; sin uso de try/except.
Botón de cancelación	Ingresa 'o' en fila o columna ejecuta break y regresa al menú principal.
Datos de pasajero inválidos	Cada función de validación tiene un while True propio que repite el input hasta recibir un dato correcto.

5.4 Guía de Uso para el Usuario

El sistema ha sido diseñado para ser intuitivo y a prueba de fallos. A continuación, se detalla el manual de usuario básico para operar la consola de **Nexus Viajes**:

1. Iniciando del Sistema

- Ejecutar el archivo principal a _main.py desde la terminal.
- El sistema desplegará automáticamente la cartelera de bienvenida (b_info.py) con los datos de la terminal y el **Menú Principal**.

2. Navegación y Comandos Básicos

- Selección de Opciones:** El menú opera mediante inputs numéricos. Se debe ingresar el número correspondiente a la acción deseada (ej: 1, 2, 3 o 4) y presionar la tecla [ENTER].
- Comando de Escape (o):** Durante el proceso de selección de asientos (Opción 2), el usuario puede ingresar el número 0 (cero) en cualquier momento para cancelar la operación de forma segura y regresar al menú principal sin alterar los datos del sistema.

5.5 Guía de Resolución de Errores

El sistema está protegido contra entradas no válidas. Si ocurre un error, el programa no se detendrá (no habrá "crasheos"), sino que guiará al usuario para corregirlo:

- **Fechas Inexistentes o Mal Formateadas:** Si en la búsqueda (Opción 1) el usuario ingresa un formato distinto a DD/MM (ej: 20-5 o Mayo 20), el sistema solicitará reingresar el dato.
 - **Letras en lugar de Números:** Si al elegir una fila o columna se ingresa texto, se mostrará un mensaje de “[ERROR] ingrese números válidos” y se reiniciará la selección.
 - **Asientos Fuera de Rango u Ocupados:** Si se intenta reservar la fila 5 (cuando el micro tiene 3) o una butaca que ya posee la marca "O", el sistema bloqueará la acción y volverá a imprimir el mapa actualizado para que el usuario elija nuevamente.
 - **Errores de Check-in:** Si el DNI contiene puntos, o el email no posee un @, el sistema retendrá la pantalla de carga hasta que el formato ingresado sea el correcto.
-

6. Conclusión

El desarrollo de la primera entrega de **Nexus Viajes** demuestra la viabilidad de construir un sistema de gestión transaccional robusto utilizando exclusivamente estructuras de datos en memoria (listas anidadas y matrices) y paradigmas de programación funcional.

Se ha logrado aislar la lógica del negocio en múltiples módulos independientes, cumpliendo con el principio de responsabilidad única. La implementación de Expresiones Regulares garantiza una alta calidad en la entrada de datos, mientras que las funciones de orden superior (filter, map, reduce) permitieron manipular la información con un código limpio, eficiente y profesional.

Este núcleo operativo (motor de ventas) se encuentra completamente estable, blindado contra errores de usuario y preparado para escalar. De esta forma, sienta las bases estructurales perfectas para la siguiente etapa del proyecto, donde se buscará implementar la persistencia de datos mediante el manejo de archivos externos.